OpsPilot: Resume Section

Sagar DS · Projects

For your resume

OpsPilot – AI helper for Linux incident checks

- Built a local AI assistant that checks server health, summarises logs and answers questions from runbooks, with no data leaving the machine.
- Used RAG (embeddings and a vector database) to find the right runbook from plain-language questions, and an AI agent that investigates alerts using only safe, read-only commands.
- Kept critical decisions in code, such as thresholds and output validation, after finding the AI unreliable with numbers.

Tech stack: Python, Ollama (Llama 3.2), ChromaDB, Prometheus, Grafana, Streamlit

Skills line

LLMs · RAG · AI agents · Prompt engineering · Vector databases · Prometheus · Grafana · Python

Profile summary line (optional)

Production operations engineer with hands-on experience applying AI (LLMs, RAG, AI agents) to monitoring and incident response.

Where to put it

List OpsPilot under **Projects**, not under your current job. It is a personal project, and that is fine: it shows initiative.

Your 2-Minute Pitch

Use this when an interviewer says *"Tell me about your project."* Say it in your own words; don't memorise it word for word.

The problem "In my job I handle production incidents. Every time, I check dashboards, read logs, find the runbook and write a summary. I wanted to see if AI could help with that, so I built OpsPilot."

The approach "It runs fully on my laptop with a local model, because in finance you can't send logs to outside AI tools."

What it does "It does four things. It reads CPU, memory and disk from Prometheus and explains whether the server is healthy. It summarises SSH logs. It answers questions from our runbooks using RAG, and if the answer isn't in the runbooks, it says so instead of guessing. And it has an agent that investigates an alert by itself: it checks health, searches the runbook and runs safe commands."

What I learnt "The biggest lesson was that the AI isn't always right. Once it said 55.7% was above a 70% threshold. So I let code make the decisions and the AI only explain them. I also limited the agent to read-only commands and a fixed number of steps."

If they ask for a demo (3 minutes)

1. **Health tab:** show the live numbers, then click *Explain health with AI*.
2. **Ask the runbooks:** ask "server is out of space", then an unrelated question to show "Not found in runbooks".
3. **Investigate:** run the agent and point out the steps it chose by itself.
4. **One honest limitation:** "With a small model, the agent sometimes repeats steps, so I added a step limit and a forced summary."

Before any interview: start the services, Ollama and the app 15 minutes early, and run each tab once so everything is loaded and fast.

Interview Q&A: The Whole Project

Short, plain answers. Learn the idea, then say it in your own words.

The project

1. What is OpsPilot? A small AI assistant for Linux incident checks. It reads server metrics, summarises logs, answers questions from runbooks and can investigate an alert on its own. Everything runs locally on a laptop.

2. Why did you build it? During incidents I spend a lot of time switching between dashboards, logs and runbooks, then writing summaries. I wanted to learn AI by solving a problem I deal with every day.

3. Why a local model instead of ChatGPT or Claude? Privacy. In companies like banks and exchanges, logs often can't be sent to external services. A local model keeps all data on the machine. It's also free to run. The trade-off is that small local models are less capable.

4. What was the hardest part? Learning not to trust the AI blindly. It made confident mistakes, such as wrong number comparisons and invented commands, so I had to design checks around it.

LLM basics

5. What is an LLM? A model trained on huge amounts of text that predicts the next word. It doesn't "know" facts the way a database does; it produces likely-sounding text, which is why it can be wrong.

6. What is a token, and why does it matter? A small piece of text, about three-quarters of a word. Models have a limit on how many tokens they can read at once (the context window), and paid APIs charge per token. That's why I never send whole log files.

7. What is temperature? A setting for randomness. Low (0 to 0.2) gives consistent answers; high gives creative ones. For operations I use 0, because the same alert should get the same answer.

8. What is hallucination? When the model confidently makes something up. Early on, mine described a Linux command that doesn't exist. I reduced this with RAG, strict prompts and code checks.

Prompts and output

9. What makes a good prompt? A clear role, the task, relevant context, the data, rules about what not to do, and the expected output format. Clear prompts give shorter, more consistent answers.

10. How did you get reliable structured output? I gave the model a JSON schema so it could only reply in a fixed shape, then validated the reply with Pydantic. If it failed, the code retried once and then showed a clear error instead of crashing.

11. How did you handle large log files? Code filters and counts first, for example failed logins per IP address. Only a short summary goes to the AI. One log went from about 41,000 tokens to under 700.

Monitoring

12. How does the monitoring part work? node_exporter publishes the machine's metrics, Prometheus collects and stores them every 15 seconds, and Grafana shows dashboards. My Python code queries Prometheus and passes the numbers to the AI.

13. Prometheus vs Zabbix? Both monitor systems. Zabbix is an all-in-one product with agents and a SQL database. Prometheus pulls metrics from exporters into its own time-series database and is the standard for Kubernetes. I use Zabbix at work and learnt Prometheus for this project.

14. Why does the code decide the health status, not the AI? The AI once said 55.7% was above a 70% threshold. Language models are unreliable with numbers, so thresholds are checked in code and the AI only explains the result.

Search and RAG

15. What is an embedding? A list of numbers that represents the meaning of a piece of text. Similar meanings get similar numbers, so you can search by meaning instead of exact words.

16. What is a vector database? A database that stores those numbers with the original text, and quickly finds the closest matches to a question. I used ChromaDB.

17. What is RAG? Retrieval-Augmented Generation. First find the relevant runbook sections, then give them to the AI together with the question, so it answers from real documents instead of memory.

18. Why RAG and not fine-tuning? Runbooks change often. With RAG, I just update the file. Fine-tuning means retraining the model every time, it's costly, and it can't show where an answer came from.

19. How do you stop RAG from making things up? The prompt tells it to answer only from the provided runbook text and to say "Not found in runbooks" otherwise. In testing, it correctly refused a question about Cisco routers.

20. How well did the search work? It found the right runbook for 4 of 5 test questions that used completely different words from the runbooks. The miss was an ambiguous question, which showed me search has to be tested, not assumed.

The agent

21. What is an AI agent? An AI that decides its own next step. It picks a tool, my code runs it, the result goes back to the AI, and it decides what to do next, until it has an answer.

22. What tools does your agent have? Check health from Prometheus, search the runbooks, and run a safe command. Only `df -h` and `uptime` are allowed.

23. How did you make the agent safe? Three guardrails, all in code: an allowlist of read-only commands, a limit of 5 steps, and a forced final summary if it doesn't finish on its own. The AI only chooses; the code decides what is allowed.

24. Did the agent work perfectly? No. With a small model, it sometimes repeated the same command and ignored a prompt telling it to stop. The step limit and forced summary handled that. It also once recommended cleanup for a healthy disk, which is why I see AI as an assistant, not a decision-maker.

Design and next steps

25. What is the main lesson from this project? A prompt is a request; code is a guarantee. Anything that must always happen, like thresholds, safety rules and validation, belongs in code. The AI is good at understanding and explaining.

26. What are the limitations? The small model is weaker at multi-step reasoning, the log analyser is tuned for one log format, and the agent has only a few tools.

27. What would you improve next? Try a larger model, connect it to Zabbix through its API, add more tools for the agent, and build a proper test set to measure answer quality over time.

28. How would this work in a real company? Use an approved model, either self-hosted or an approved cloud service. Load the team's real runbooks, connect it to the existing monitoring, keep the agent read-only, and keep a human in charge of every action.